Software Test Plan (STP)

Change Management Software

Project	Change Management Software (CMS)
Version	2.0 (as built)
Authors	Team 4: Vivaan Atharva (PES1UG24AM071), Dhanush S S (PES1UG24AM082), Gayathri Priya V (PES1UG24AM105), Gonu Guntla Hema Kowshik (PES1UG24AM106)
Date	7 November 2026
Status	Final: describes the release built in Sprints 1 and 2 (feature freeze 7 Nov 2026)

Revision History

Version	Date	Author	Change summary	Approval
1.0	28 October 2026	Team 4	Initial project-specific draft	Team
2.0	7 November 2026	Team 4	Updated to the implemented system: as-built technology, design and test results; implementation status and deviations recorded	Team

Approvals

Role	Name	Signature / Email	Date
Team lead	Vivaan Atharva		
Course instructor / TA	To be filled by instructor	Pending	

Table of Contents

- 1. Introduction
- 2. Test Items
- 3. Features to be Tested
- 4. Features Not to be Tested
- 5. Test Approach / Strategy
- 6. Test Environment
- 7. Test Schedule
- 8. Test Deliverables
- 9. Roles and Responsibilities
- 10. Risks and Mitigation
- 11. Assumptions and Dependencies
- 12. Suspension and Resumption Criteria
- 13. Test Case Management and Traceability
- 14. Test Metrics and Reporting
- 15. Approvals
- 16. Test Summary Report (end of Sprint 2)

1. Introduction

1.1 Purpose

This plan defines the scope, strategy, environment, responsibilities, and exit criteria for testing CMS. Tests verify that change requests are created and stored correctly, the status workflow allows only valid transitions, assignments and resolutions are recorded with full history, overdue requests are flagged accurately, and access controls are enforced. Version 2.0 records the actual environment, schedule and responsibilities, the result of every planned test case (section 13) and the final test summary (section 16).

1.2 Scope

Testing covers authentication/authorization, change request creation and editing, field validation, workflow transitions, approval and rejection, assignment and reassignment, comments and work notes, resolution, closure and reopening, status history, SLA/overdue detection, notifications, dashboard and reports, audit events, and key non-functional requirements. Email/chat delivery, source-control or CI/CD integration, and production-scale infrastructure are excluded unless implemented.

2. Test Items

- Authentication and role authorization
- Change request management module
- Workflow engine (status transitions)
- Assignment module
- Comments and activity timeline
- SLA / overdue detection
- Notification module
- Dashboard and reporting module
- Audit history
- Application routes, services, and persistence layer

3. Features to be Tested

Requirement / feature	Test focus
CMS-F-001 to CMS-F-002	Valid/invalid login, logout, role restrictions, direct protected-

	operation access.
CMS-F-003 to CMS-F-006	Create, view, edit, search, filter, and validation of CR fields; edits blocked on Closed CRs.
CMS-F-007 to CMS-F-008	Every allowed and disallowed workflow transition; rejection requires a reason.
CMS-F-009 / CMS-SR-005	Assignment to active engineers only; reassignment; engineers limited to their own CRs.
CMS-F-010 to CMS-F-012	Start work, work notes and comments, resolve with summary, close, and reopen with reason.
CMS-F-013	History rows for each status and assignment change, in order, with actor and timestamp.
CMS-F-014	Due-date calculation and overdue classification at boundary values.
CMS-F-015	Notifications created for assignment, status change and comment events.
CMS-F-016 to CMS-F-018	Dashboard counts, open-CR report, ageing and resolution-time report contents.
CMS-F-019 / CMS-SR-006	Audit event content and exclusion of secrets.
CMS-NF-001	Dashboard/search response time on the agreed test dataset.
CMS-NF-002 / CMS-NF-006	Persistence after restart and documented setup on a clean environment.
CMS-NF-003 / CMS-SR-001 to CMS-SR-004	Credential storage, authorization, validation, output escaping and safe error responses.
CMS-NF-007	Concurrent edits to the same CR are detected and not silently overwritten.

4. Features Not to be Tested

- Whether the change described in a CR was actually carried out in an external system; the system relies on user-entered data.
- Email, chat, source-control, CI/CD or other external integrations that are not part of the submitted build.
- Production infrastructure availability and load at enterprise scale.
- Third-party database and framework internals; test only the application's use of them.

5. Test Approach / Strategy

5.1 Test Levels

- Unit testing: validate service-level rules such as field validation, the workflow transition table, assignment eligibility, due-date calculation and overdue classification.
- Integration testing: validate service/repository/database interactions, transactional status and history writes, and optimistic locking.
- System testing: run end-to-end workflows (create, approve, assign, resolve, close, reopen) through the application interface or API.
- Acceptance testing: demonstrate the main user stories against the submitted SRS and verify the working product.

5.2 Test Types

- Functional testing of each implemented requirement.
- Negative and boundary-value testing for required fields, invalid transitions, permissions, due dates and empty data.
- State-transition testing covering every pair of statuses in the workflow table.
- Regression testing after service, UI, or schema changes.
- Security checks for authentication, authorization, record-level access, password storage, input validation, output escaping, and logs.
- Performance checks for dashboard/search using a documented dataset.
- Usability checks for clear navigation, labels, validation feedback, and status indicators.

5.3 Entry Criteria

- A runnable build is available and setup steps are documented.
- Test environment and test database are available.
- The team has agreed the implemented requirement set, the workflow transition table, and test data.
- Critical dependencies are installed and automated tests can be executed.

5.4 Exit Criteria

- All planned high-priority test cases have been executed.
- All critical defects are fixed or explicitly accepted by the team/instructor; no known critical workflow, history-integrity or authorization defect remains.
- Every test CR's history is consistent with its current status.
- Implemented requirements are linked to test cases in the RTM.
- Test results and deviations from SRS/SAD are documented.

6. Test Environment

Area	As used
Hardware	Team laptops: two macOS and two Windows machines; GitHub-hosted runners for CI.
Software	Python 3.12, Flask 3, Flask-SQLAlchemy 3.1, Flask-Login, SQLite; versions from requirements.txt.
Browser	Chrome and Safari (current versions) for manual system checks.
Tools	pytest; GitHub Actions running the full suite on Ubuntu, Windows and macOS for every pull request and every push to main; Jira for stories; GitHub pull requests for review.
Test data	pytest fixtures with an in-memory SQLite database per test. The seeded data set has 7 users (one per role, two engineers, an inactive engineer, two requesters) and 10 CRs covering every status and priority, with due dates in the past, today and the future. Boundary fixtures cover due yesterday, today and tomorrow.
Isolation	Every test builds a fresh in-memory database, so tests never touch the developer's local data.

7. Test Schedule

Activity	Actual dates	Output
Prepare SRS, SAD and this plan	28-30 Oct 2026	Version 1.0 documents
Sprint 1: unit and integration tests with each story	31 Oct - 1 Nov 2026	Login, create, search, workflow engine, assignment, dashboard; CI green on 3 OSes
Sprint 2: lifecycle stories and end-to-end tests	2 - 7 Nov 2026	Review, start/resolve, close/reopen, edit, due dates; integration suite run through the real pages
Final regression and documents	7 Nov 2026 (feature freeze)	354 passing tests, final RTM, version 2.0 documents
Demonstration	9 Nov 2026	Live demo of the full lifecycle

8. Test Deliverables

- This Software Test Plan.
- Test cases and test data/fixtures.
- Automated unit and integration test scripts.
- Test execution logs and CI results.
- Defect/issue records with reproduction steps.
- Requirements Traceability Matrix (RTM).

- Final Test Summary Report including pass/fail counts, known issues, and deviations.

9. Roles and Responsibilities

Team member	Stories (Jira)	Testing responsibilities
Vivaan Atharva (team lead)	CMS-1 setup, CMS-2 login, CMS-12 dashboard, CMS-14 notifications, CMS-15 integration tests and documents	CI pipeline, end-to-end integration tests, RTM, test summary; reviews and merges pull requests.
Dhanush S S	CMS-3 create, CMS-4 list/search, CMS-11 edit, CMS-13 due dates	Tests for validation, search and filters, edit rules and due-date boundaries.
Gayathri Priya V	CMS-5 workflow engine, CMS-6 approve/reject, CMS-8 start/resolve	Transition-table tests (every allowed and disallowed pair) and review/resolve route tests.
Gonu Guntla Hema Kowshik	CMS-7 assignment, CMS-9 close/reopen, CMS-10 comments	Assignment eligibility, reassignment and close/reopen tests.

Every pull request needs green CI on all three operating systems and a review before it is merged into main.

10. Risks and Mitigation

Risk	Mitigation
Late integration causes failures	Merge small PRs frequently and run CI on every PR.
Workflow tests miss a transition	Generate test cases from the transition table so every allowed and disallowed pair is covered.
Test data does not cover due-date boundaries	Include CRs due yesterday, today, tomorrow, and with no SLA set.
History drifts from current status	Use transactional service logic and a history-consistency test.
Tests depend on local configuration	Document environment setup and keep secrets out of the repository.
Requirements changed during implementation	Record each approved deviation and update SRS, RTM, and test cases.

11. Assumptions and Dependencies

- The project build and dependencies can be installed using the repository instructions.
- Test data is synthetic and can be reset between runs.
- All four team members have access to the shared repository and can review/merge changes according to the team's workflow.
- Any external integration not present in the build is treated as out of scope.

12. Suspension and Resumption Criteria

Suspend a test run if the application cannot start, the test database is unavailable, a blocker prevents more than 30% of planned cases from running, or a test risks corrupting shared data. Resume when the blocking defect is fixed, the environment is restored, and a smoke test passes.

13. Test Case Management and Traceability

Result at the feature freeze. "Pass" means automated tests covering the case pass on all three CI operating systems; the test file is named in brackets. "Not run" cases test features that were not built.

Test case ID	Scenario	Requirement	Result
TC-AUTH-01	Sign in with valid credentials	CMS-F-001	Pass (test_auth)
TC-AUTH-02	Protected action without permission	CMS-F-002, SR-002	Pass (test_auth, test_integration)
TC-CR-01	Create CR with valid fields; visible in search	CMS-F-003	Pass (test_change_requests, test_integration)

TC-CR-02	Create CR with missing title	CMS-F-003, F-021	Pass (test_change_requests)
TC-VAL-01	Priority or component that does not exist	CMS-F-004	Pass (test_change_requests)
TC-VAL-02	Script text in description shown as plain text	CMS-SR-004	Pass (test_integration)
TC-WF-01	Full lifecycle Submitted to Closed	CMS-F-007	Pass (test_workflow, test_integration)
TC-WF-02	Submitted directly to Closed	CMS-F-007	Pass (test_workflow, test_integration)
TC-WF-03	Reject without a reason	CMS-F-008	Pass (test_review, test_integration)
TC-ASSIGN-01	Assign approved CR to active engineer	CMS-F-009, F-015	Pass (test_assignment, test_notifications)
TC-ASSIGN-02	Assign to inactive user or non-engineer	CMS-F-009	Pass (test_assignment)
TC-ASSIGN-03	Engineer resolves someone else's CR	CMS-SR-005	Pass (test_start_resolve, test_integration)
TC-RES-01	Resolve without a summary	CMS-F-011	Pass (test_start_resolve, test_integration)
TC-CLOSE-01	Close a Resolved CR; edits then blocked	CMS-F-012, F-006	Pass (test_close_reopen, test_edit_change_request, test_integration)
TC-REOPEN-01	Reopen with a reason; assignee notified	CMS-F-012, F-015	Pass (test_close_reopen, test_notifications)
TC-HIST-01	Full lifecycle with one reassignment; history in order	CMS-F-013	Pass (test_integration)
TC-SLA-01	Open CR due yesterday is overdue	CMS-F-014	Pass (test_due_dates, test_dashboard)
TC-SLA-02	Resolved CR past due is not overdue	CMS-F-014	Pass (test_due_dates, test_integration)
TC-REP-01	Open-CR report totals	CMS-F-016, F-017	Dashboard part: Pass (test_dashboard); report: not run
TC-REP-02	Resolution-time report	CMS-F-018	Not run (not built)
TC-AUDIT-01	Approve and assign; actor and time recorded	CMS-F-019, SR-006	Pass for status and assignment history (test_integration)
TC-CONC-01	Two users edit the same CR	CMS-NF-007	Not run (not built)
TC-PERF-01	Dashboard/search timing on 10,000 CRs	CMS-NF-001	Not run
TC-SEC-01	No plaintext passwords stored or logged	CMS-NF-003, SR-003	Pass (test_project_setup, test_auth)
TC-RESTART-01	Data survives an application restart	CMS-NF-002	Pass (test_integration)

14. Test Metrics and Reporting

- Execution progress = executed test cases / planned test cases × 100%.
- Pass rate = passed test cases / executed test cases × 100%.
- Track failed, blocked, and not-run cases separately.
- Track open defects by severity, age, and affected requirement.
- Report requirement coverage as requirements with at least one test / implemented requirements.
- Report workflow coverage as transitions tested / transitions in the workflow table.
- Publish daily or per-PR status during the sprint and a final test summary before the demo.

15. Approvals

Role	Name	Signature / Date
Test coordinator / QA	To be filled by team	
Developer representative	To be filled by team	
Project representative	To be filled by team	
Instructor / TA	To be filled if required	

16. Test Summary Report (end of Sprint 2)

Measure	Result
Automated tests	354 passed, 0 failed, on each of Ubuntu, Windows and macOS (GitHub Actions, Python 3.12, 7 Nov 2026)

Planned test cases (section 13)	25 planned; 21 passed in full, 1 passed for the dashboard part only, 3 not run because the feature was not built or not measured
Workflow coverage	Every allowed transition in the workflow table and every disallowed pair of statuses is tested (test_workflow)
Requirement coverage	30 of the 34 SRS requirements are implemented in full or in part; every one of them has at least one automated test (see SRS section 9 and docs/RTM.md)
Open defects	None known at the feature freeze
Not tested	Comments, reports, concurrent edits and the performance target (features not built; see SRS section 10)

16.1 Test Suites

Test file	What it checks
test_project_setup	App start-up, health check, password hashing, seed data, 404 page
test_auth	Sign in/out, inactive users, safe redirects, 403 for wrong roles, CSRF
test_change_requests, test_change_request_search	Create and validate; list, search, filters, sort, record-level visibility
test_workflow	The transition table, required inputs, history rows, closed_at
test_review, test_start_resolve, test_close_reopen	Workflow routes and buttons for each role
test_assignment	Assign and reassign rules, assignment history, 403s
test_edit_change_request, test_due_dates	Edit rules and history; SLA due dates and overdue boundaries
test_dashboard	Counts, work lists per role, overdue list
test_notifications	Who is notified for each event, never the actor, no notification on a refused action, unread count, list, open and mark all read, privacy
test_integration	End-to-end lifecycle through the real pages as requester, reviewer and engineer; reopen, reject, reassign, access control, escaping, restart

Exit criteria (section 5.4) are met for the implemented scope: all high-priority cases for built features pass, no critical defect is open, history matches status in every lifecycle test, and the RTM links every implemented requirement to tests. Deviations from the SRS are recorded in SRS section 10.